

MetKnight - Leadership Pitch

The One-Liner

MetKnight is Dependabot's coverage + Sonar/Veracode's inline developer experience + Copilot's remediation power - running locally, for free, inside the IDE developers already live in.

1. The Problem Today

Dependabot/Veracode/Sonar findings live in a dashboard developers don't check - they surface post-commit, in PRs, or in a separate portal, days after the code was written.

Fixing a flagged CVE today means: Open ticket -> find the file -> figure out the patched version -> bump it -> hope nothing breaks -> wait for CI -> repeat per package.

Multiply that by hundreds of repos and thousands of transitive dependencies - this is pure developer time and org risk, not license cost (those tools are already paid for), but the **time-to-remediate is where the real cost sits.**

2. What MetKnight Does

Scans **npm, Gradle, Maven, and pip manifests live, in the editor**, against an **offline GHSA mirror** - no per-scan network/API cost.

Flags CVEs and **n-2 version staleness inline** - same UX pattern as SonarLint squiggles.

"Fix All" - the differentiator: one click resolves every flagged CVE automatically.

Mechanically bumps the manifest to the patched version - **deterministic, instant, zero AI cost.**

Actually runs the project's real **build/install** to prove it didn't break anything.

Only escalates to **Copilot** when the build genuinely fails or a fix needs judgment - e.g. transitive dependency or breaking API change - and hands Copilot the exact failure log so it fixes forward instead of rediscovering the problem.

Every dependency removal requires **explicit developer approval - no silent breakage.**

3. Cost / Time / Token Savings

Token cost: the old approach sent every fix to Copilot in agent mode - multiple tool calls, file reads, build retries per package. The new mechanical-first pipeline means the common case - a clean version bump - costs **zero LLM tokens**. Copilot is only invoked for the genuinely hard cases.

Time: what used to be a manual multi-step remediation - find -> research -> bump -> test -> repeat - is now a **single click**, seconds to minutes instead of the hours typically spent per vulnerability batch.

Effort/tool removed: no ticket triage, no manual patched-version lookup, no manual dependency-tree spelunking to find why a transitive package is even present. MetKnight now traces and displays the **parent dependency automatically.**

Net effect: this is the same value prop as "shift-left security," but actually acted upon at write-time instead of just reported.

4. How It Stacks Up

Capability	Dependabot	SonarQube	Veracode	MetKnight
Inline IDE feedback	No	Yes (lint-only)	No	Yes
CVE/advisory detection	Yes	Partial	Yes	Yes (offline mirror)
n-2 staleness / drift detection	No	No	No	Yes
One-click automated remediation	No (PR only)	No	No	Yes

Capability	Dependabot	SonarQube	Veracode	MetKnight
Verifies the fix actually builds	No	No	No	Yes
Transitive dependency traceability	Limited	No	No	Yes
Runs without sending code to 3rd-party SaaS	No	No	Depends	Yes (offline-first)
Cost per fix	PR review time	N/A	N/A	Seconds, near-zero token cost

MetKnight isn't replacing these tools' compliance/reporting function - it's the missing "last mile" that turns their findings into closed tickets automatically.

5. Scope for Expansion

Multi-IDE: The core engine - advisory matching, version resolution, mechanical-bump-then-build logic - is already IDE-agnostic TypeScript/Node. A JetBrains (IntelliJ/WebStorm) plugin is a UI-layer port, not a rewrite, since the heavy lifting isn't VS Code-specific.

Ecosystem coverage: npm/Gradle/Maven/pip today; **NuGet (.NET) and Go modules** are natural next additions given the same parser pattern.

Org-wide policy enforcement: the existing policy-as-code engine - block-prerelease, blocklist, min-version rules - can become a centrally-pushed compliance baseline. Security teams define the rules once, every repo/IDE enforces them automatically.

CI/CD gate: the same mechanical-bump-and-build logic can run headless in a pipeline stage, turning this into a **pre-merge gate**, not just a developer convenience.

Fleet-wide dashboard: aggregate the per-repo output-channel/report data into a rollup view - "how stale is our dependency posture org-wide," a metric leadership can actually track quarter over quarter.

6. The Ask

Position this as an internal force-multiplier on top of existing security spend.

Dependabot/Veracode/Sonar stay as the system of record and compliance layer - MetKnight is the remediation accelerator that closes the loop.